

T08 — Costruttori, Inizializzazione ed Incapsulamento

Ciclo di Inizializzazione, Overloading dei Costruttori, Modificatori e SimpleDate v3

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo in dettaglio la teoria dei costruttori, dell'incapsulamento e dell'information hiding ([T08 - Class Constructors and Encapsulation.pdf](#)), confrontandola con il codice ufficiale in [Lezione08](#), dove nasce l'enum `Language.java`, vengono aggiunti gli array dei mesi per la stampa estesa (`prettyPrint()`) e si introduce l'**Enhanced For** (`for-each`).

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T08)

A. Il Ciclo di Vita e i Meccanismi dei Costruttori (Slide 1–12)

- **Cosa avviene esattamente dietro le quinte con `new Class()`:**
 1. La JVM calcola l'ingombro in byte di tutti i campi e alloca un blocco contiguo di memoria nello **Heap**.
 2. Genera un riferimento (*reference*) univoco che punta a quell'area di memoria.
 3. Inizializza tutti i campi ai rispettivi valori di default (`0`, `false`, `null`).
 4. Invoca il **costruttore** designato passando implicitamente il puntatore `this`.
 5. Restituisce l'indirizzo dell'oggetto alla variabile a sinistra dell'assegnamento.
- **Caratteristiche Tassative di un Costruttore:**
 - Deve avere lo **stesso identico nome della classe**.
 - **Non ha alcun tipo di ritorno** (nemmeno `void`!).
 - Viene eseguito una sola volta, all'atto della creazione dell'istanza.
- **Default Constructor (Costruttore di Default):**
 - Se il programmatore **non dichiara alcun costruttore**, il compilatore Java inserisce automaticamente un costruttore vuoto senza argomenti (`public Class(){ }`).
 - Attenzione: **Regola aurea d'esame:** se il programmatore dichiara *anche un solo* costruttore con parametri (es. `public Car(String color)`), il compilatore **non genera più il costruttore di default!** Invocare `new Car()` senza argomenti provocherà un **Compile-Time Error**.
- **False Constructor (Il Falso Costruttore — Trabocchetto d'Esame):**
 - Se si inserisce un tipo di ritorno (ad esempio `void Car(String color)`):

```
1 public class Car {
2     void Car(String color) { ... } // Attenzione: NON È UN COSTRUTTORE!
3 }
```

- Per il compilatore questo è un **normale metodo d'istanza** (avente incidentalmente lo stesso nome della classe). Il compilatore aggiungerà comunque il costruttore di default `Car()`, e il metodo `void Car(...)` non verrà mai invocato al momento del `new`!
- **Copy Constructor:**
 - Costruttore che accetta un'altra istanza della medesima classe per clonarne lo stato (`Car(Car other)`). In Java non esiste la copia automatica bit-a-bit del C++: deve essere scritta esplicitamente dal programmatore.
- **Constructor Overloading e Chaining con `this(...)`:**
 - Più costruttori con firme distinte.
 - La chiamata `this(...)` verso un costruttore fratello serve a evitare duplicazione di logica di validazione e **deve essere tassativamente la prima riga di codice nel corpo del costruttore**.
 - *Meme delle slide (p. 11):* Il matematico in lacrime che grida “*Abuse of notation*” vs il Chad Programmer che usa l'overloading per semplificare il codice.
- **Distruttori e il Metodo `finalize()`:**
 - In Java non esiste la deallocazione manuale: non esistono distruttori (`~Car()`).
 - Esisteva il metodo protetto `protected void finalize()` invocato prima della rimozione da parte del Garbage Collector.
 - Attenzione: **Attenzione:** `finalize()` è ufficialmente **deprecato da Java 9** in poi (e rimosso/reso no-op nelle versioni moderne) per l'imprevedibilità temporale dell'esecuzione del GC e rischi di deadlock. Non va mai utilizzato.

B. Incapsulamento, Information Hiding e Modificatori di Accesso (Slide 13–24)

- **Incapsulamento:** aggregazione fisica nello stesso file sorgente di dati (campi) e procedure (metodi).
- **Information Hiding:** occultamento della rappresentazione interna dello stato.
 - *Meme della Cipolla (p. 18):* «*Most software is written like an onion: The more layers you peel back, the more you want to cry*». L'Information Hiding crea confini netti per proteggere il codice dal disfacimento.
 - *Meme di Gandalf (p. 24):* «*YOU SHALL NOT ACCESS MY MEMBERS*».
- **I 4 Livelli di Visibilità in Java:**
 1. `private`: visibile esclusivamente all'interno della stessa classe.
 2. *Default (Package-Private)*: nessun modificatore; visibile all'interno della classe e a tutte le classi residenti nello stesso package.
 3. `protected`: visibile nel package e a tutte le sottoclassi derivate (anche in package diversi).
 4. `public`: visibile ovunque, da qualsiasi package del Classpath.
- **Vantaggi Architetturel di Getter e Setter:**
 - *Nei Setter:* possibilità di intercettare valori anomali, applicare regole di business e loggare le modifiche prima di mutare lo stato.
 - *Nei Getter:* possibilità di alterare la rappresentazione interna della memoria senza rompere il codice dei client (**Retro-compatibilità / Loose Coupling**). Ad esempio, cambiare il campo interno da `int age` a `short age` per dimezzare l'occupazione di RAM, mantenendo il getter `public int getAge(){ return age; }` (promozione implicita di tipo) senza che il mondo esterno debba ricompilare o cambiare una sola riga di codice.

C. Asserzioni e Testing (Slide 25–26)

- La parola chiave `assert`:

```
1 assert person.getAge() != 0 : "Age cannot be 0";
```

- Se l'espressione a sinistra valuta `false`, la JVM interrompe l'esecuzione e lancia un `AssertionError` mostrando il messaggio a destra dei due punti.
- Richiede obbligatoriamente l'attivazione a runtime con il flag JVM `-ea` (*Enable Assertions*):

```
1 java -ea TestPerson
```

0.0.2 2. Codice: Evoluzione del Codice: [Lezione08](#)

In [Lezione08](#) il docente applica un refactoring strutturale a `SimpleDate`:

1. Introduzione del Tipo Enumerato `Language.java` Viene abbandonato il primitivo `byte lang` (dove 0 era IT e 1 era US, fragile e poco leggibile) a favore di un tipo enum fortemente tipizzato:

```
1 public enum Language {
2     IT, US;
3
4     public String toString() {
5         switch (this) {
6             case IT: return "italiano";
7             case US: return "american";
8             default: return null;
9         }
10    }
11 }
```

Finezza: Anche gli enum in Java sono tipi reference completi e possono sovrascrivere `toString()` per restituire una descrizione testuale appropriata.

2. Refactoring di `Date.java`

```
1 public class Date {
2     - private byte lang; // 0: IT, 1: US
3     + private Language lang;
4     + private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
5     + private static final String[] MONTHS_US = { "January", "February", "March", "April", "May",
6
7     public Date(int day, int month, int year) {
8         this.day = day;
9         this.month = month;
10        this.year = year;
11        verify();
12        - lang = 0;
13        + lang = Language.IT;
14    }
15
16    - public void setLang(byte lang) { ... }
17    + public void setAmerican() { lang = Language.US; }
18    + public void setItalian() { lang = Language.IT; }
19 }
```

```

20 + public String getMonthAsString() {
21 +     if (lang == Language.IT) return MONTHS_IT[month-1];
22 +     else return MONTHS_US[month-1];
23 + }
24
25 + public String prettyPrint() {
26 +     if (lang == Language.IT) return day + " " + MONTHS_IT[month-1] + " " + year;
27 +     else return MONTHS_US[month-1] + " " + day + ", " + year;
28 + }

```

Analisi: Dettagli Tecnici e Trabocchetti nel Codice di Date:

1. **Separazione Semantica dei Metodi di Configurazione:** Invece di un setter generico che accetta valori arbitrari, il prof definisce due metodi espliciti auto-esplicativi: `setItalian()` e `setAmerican()`.
2. **Array di Costanti di Classe (`private static final String[]`):** Gli elenchi dei nomi dei mesi sono `static final` (condivisi da tutte le istanze senza duplicare memoria nello Heap) e `private` (inaccessibili all'esterno). *Curiosità nel codice del prof:* nell'array `MONTHS_IT` c'è un piccolo refuso: "setembre" con una sola 't'.
3. **Persistenza del Refuso del Prof nei Getter:** Anche in questa lezione, alle righe 43–44 di `Date.java`:

```

1 public int getDay() { return day; }
2 public int getMonth() { return day; } // Ritorna day anziché month!
3 public int getYear() { return day; } // Ritorna day anziché year!

```

Un chiaro refuso di copia-incolla che non è stato corretto dal docente nel rilascio ufficiale. Nel proprio codice personale è doveroso restituire i rispettivi campi `month` e `year`.

4. `prettyPrint()` e Formattazione Culturale:

- In italiano: giorno, mese per esteso, anno (es. 7 gennaio 2025).
- In inglese/americano: mese per esteso, giorno, virgola, anno (es. February 20, 2025).

3. Novità in `MainDate.java`: Array di Oggetti ed Enhanced For

```

1 // Inizializzazione rapida di un array di oggetti nello Heap
2 Date[] dates = { d1, d2, d3, new Date(14, 2, 2024) };
3
4 // 1. Scansione all'indietro classica tramite indice
5 for (int i = dates.length - 1; i >= 0; i--)
6     System.out.println(dates[i].toString() + ": " + dates[i].getMonthAsString());
7
8 // 2. Scansione in avanti con Enhanced For (For-Each)
9 for (Date date : dates)
10     System.out.println(date.toString() + ": " + date.getMonthAsString());

```

- Il costrutto `for (Tipo elemento : collezione)` evita di manipolare manualmente indici di scorrimento, rendendo il codice più leggibile ed eliminando il rischio di errori di off-by-one o out-of-bounds.

0.0.3 3. Ciclo: Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository `esercizi-wally-25-26`: - **es06:** `Person.java` implementa l'esempio accademico della persona con i metodi di validazione `verifyAge` e `verifyName`, incapsulamento rigoroso e gestione dei setter/getter. - **es07:** `Language.java` definisce l'enum `Language` sfruttando modernamente la *Switch Expression* di Java 14+ (case `IT -> "italiano"`).

Nota di Studio

Con la **Lezione 08** abbiamo completato l'intero ciclo di vita dei costruttori, l'incapsulamento dei dati, gli enum base e la gestione dei formati estesi.

Il prossimo blocco è la **Lezione 09 / Slide T09: Arrays and Enumerative Types**: - Approfondimento teorico sistematico sugli **Array in Java** (dichiarazione, allocazione nello Heap, proprietà immutabile `.length`, array multidimensionali, copie superficiali vs profonde). - Analisi rigorosa degli **Enumerative Types (enum)**: costruttori di enum, campi interni, metodi `values()` e `ordinal()`. - Evoluzione del codice in Lezione09: - Potenziamento di `Language.java` con campi interni (`format`, `description`) e costruttore privato. - Riscrittura ed espansione di `Date.java`.

Dammi conferma per aprire la Lezione 09 / T09 e continuare!